

온라인 쇼핑몰 프로젝트 구현 계획 초안

0. 문서 목적

이 문서는 Spring Boot 기반 온라인 쇼핑몰 프로젝트를 실제 구현하기 위한 1차 실행 계획서이다.

앞선 설계 문서에서는 온라인 쇼핑몰을 단순 CRUD 서비스가 아니라, 주문·결제·재고의 정합성을 중심으로 운영 가능한 시스템으로 정의했다.

이제 필요한 것은 전체 아키텍처를 한 번에 모두 구현하는 것이 아니라, 구현 가능한 범위로 나누고 핵심 기능부터 검증하는 것이다.

이 문서의 목적은 다음과 같다.

1. 프로젝트의 1차 MVP 범위를 정의한다.
2. 구현 우선순위를 정한다.
3. 먼저 작성해야 할 Entity, Repository, Service, Controller를 정리한다.
4. 주문·결제·재고 흐름을 실제 코드로 옮기기 위한 기준을 세운다.
5. 동시성 테스트와 성능 테스트까지 포함한 실무형 구현 방향을 정리한다.

1. 현재 설계 상태

현재까지 정리한 산출물은 다음과 같다.

1. 온라인 쇼핑몰 운영 아키텍처 문서
2. ERD 상세 설계서
3. API 명세서
4. 시퀀스 다이어그램 설계서
5. 주문 상태 머신
6. 결제 상태 머신
7. 재고 예약/차감 전략
8. 동시성 제어 전략
9. 배치 및 장애 복구 전략
10. 운영 모니터링 기준

현재 설계는 운영 가능한 쇼핑몰 전체 구조를 대상으로 한다.

하지만 실제 구현은 전체 범위를 한 번에 진행하지 않는다.
먼저 핵심 도메인을 구현하고, 이후 확장 도메인을 추가한다.

2. 프로젝트 목표

2-1. 최종 목표

이 프로젝트의 최종 목표는 다음과 같다.

회원이 상품을 조회하고,
상품을 주문하고,
결제를 완료하고,
재고가 정확히 차감되며,
주문 상태와 결제 상태가 일관되게 유지되는
Spring Boot 기반 쇼핑몰 주문 시스템을 구현한다.

단순히 상품 CRUD를 만드는 것이 목적이 아니다.

핵심은 다음과이다.

1. 동시에 주문해도 재고가 음수가 되지 않는다.
2. 결제 성공 후 주문 상태가 누락되지 않는다.
3. 결제 실패 시 예약 재고가 복구된다.
4. 주문 조회에서 N+1 문제가 발생하지 않도록 최적화한다.
5. 동시성 테스트로 정합성을 검증한다.
6. 성능 테스트로 인덱스와 페이징 효과를 확인한다.

3. 1차 MVP 범위

3-1. 포함할 기능

1차 구현 범위는 다음으로 제한한다.

회원

- 회원가입
- 로그인 또는 임시 인증 처리
- 내 정보 조회

상품

- 상품 등록
- 상품 목록 조회
- 상품 상세 조회

재고

- inventory 테이블 분리

- 조건부 UPDATE 기반 재고 예약
- 결제 성공 시 예약 재고 확정
- 결제 실패 시 예약 재고 해제

주문

- 주문 생성
- 주문상품 저장
- 배송정보 저장
- 내 주문 목록 조회
- 주문 상세 조회
- 주문 취소

결제

- 실제 PG 연동은 제외
- Mock 결제 승인 API 구현
- 결제 성공 처리
- 결제 실패 처리

리뷰

- 구매한 상품에만 리뷰 작성
- 하나의 order_item에는 하나의 리뷰만 작성 가능

3-2. 1차 MVP에서 제외할 기능

다음 기능은 1차 구현에서 제외한다.

- 실제 PG사 연동
- 실제 배송사 API 연동
- 쿠폰
- 포인트
- 정산
- Redis 캐시
- Message Queue
- Outbox 패턴
- 관리자 고급 검색
- 상품 대량 업로드
- 배치 재실행 관리 화면

이 기능들은 중요하지만, 처음부터 모두 구현하면 프로젝트 범위가 과도하게 커진다.

1차 목표는 주문·결제·재고 정합성의 핵심 흐름을 먼저 완성하는 것이다.

4. 구현 우선순위

구현 순서는 다음과 같이 진행한다.

- 1단계. 프로젝트 기본 세팅
- 2단계. 핵심 Entity 작성
- 3단계. Repository 작성
- 4단계. 도메인 로직 작성
- 5단계. 주문 생성 Service 구현
- 6단계. 주문 생성 API 구현
- 7단계. Mock 결제 성공/실패 API 구현
- 8단계. 주문 조회 API 구현
- 9단계. 리뷰 작성 API 구현
- 10단계. 동시성 테스트 작성
- 11단계. 조회 성능 최적화
- 12단계. README 및 포트폴리오 문서 정리

핵심은 Controller부터 만들지 않는 것이다.

먼저 Entity와 도메인 로직을 만들고, 그 다음 Service 유스케이스를 구현한다.

5. 프로젝트 패키지 구조 초안

패키지 구조는 도메인 기준으로 나눈다.

```
src/main/java/com/example/shop

member
├─ controller
├─ service
├─ repository
├─ domain
└─ dto

product
├─ controller
├─ service
├─ repository
├─ domain
└─ dto

inventory
├─ service
└─ repository
```

```
|— domain
└─ dto
```

order

```
|— controller
|— service
|— repository
|— domain
└─ dto
```

payment

```
|— controller
|— service
|— repository
|— domain
└─ dto
```

delivery

```
|— domain
└─ repository
```

review

```
|— controller
|— service
|— repository
|— domain
└─ dto
```

global

```
|— config
|— exception
|— security
|— common
└─ response
```

초기에는 패키지를 너무 복잡하게 나누지 않는다.
하지만 주문, 결제, 재고는 반드시 분리한다.

이유는 세 도메인의 책임이 다르기 때문이다.

```
Order      → 주문 상태와 주문상품 관리
Payment    → 결제 상태와 승인/실패 처리
Inventory  → 재고 예약, 확정, 해제 처리
```

6. 1차 테이블 구현 범위

처음 만들 테이블은 다음과 같다.

```
member
category
product
inventory
orders
order_item
payment
delivery
review
product_stat
```

추후 확장할 테이블은 다음과 같다.

```
cart
cart_item
coupon
coupon_issue
point_history
outbox_event
order_event_log
batch_job_execution
batch_failure
```

7. 핵심 Entity 구현 순서

Entity는 다음 순서로 작성한다.

1. Member
2. Category
3. Product
4. Inventory
5. Order
6. OrderItem
7. Delivery
8. Payment
9. Review
10. ProductStat

Entity 작성 시 주의할 점은 다음과 같다.

1. Entity를 API 응답으로 직접 반환하지 않는다.
2. Setter를 무분별하게 열지 않는다.
3. 상태 변경 메서드는 Entity 내부에 둔다.
4. 생성 로직은 정적 생성 메서드로 정리한다.
5. 연관관계 편의 메서드를 작성한다.

예시:

```
public class Order {

    public static Order createPending(Member member) {
        Order order = new Order();
        order.member = member;
        order.status = OrderStatus.PENDING_PAYMENT;
        order.orderedAt = LocalDateTime.now();
        return order;
    }

    public void pay() {
        if (this.status != OrderStatus.PENDING_PAYMENT) {
            throw new IllegalStateException("결제 대기 상태에서만 결제 완료가 가능합니다.");
        }
        this.status = OrderStatus.PAID;
        this.paidAt = LocalDateTime.now();
    }

    public void cancel() {
        if (this.status == OrderStatus.SHIPPED || this.status == OrderStatus.DELIVERED) {
            throw new IllegalStateException("배송 시작 이후에는 일반 취소가 불가능합니다.");
        }
        this.status = OrderStatus.CANCELLED;
        this.cancelledAt = LocalDateTime.now();
    }
}
```

8. 가장 먼저 구현할 핵심 유스케이스

8-1. 주문 생성

가장 먼저 구현할 기능은 주문 생성이다.

주문 생성은 단순 INSERT가 아니다.

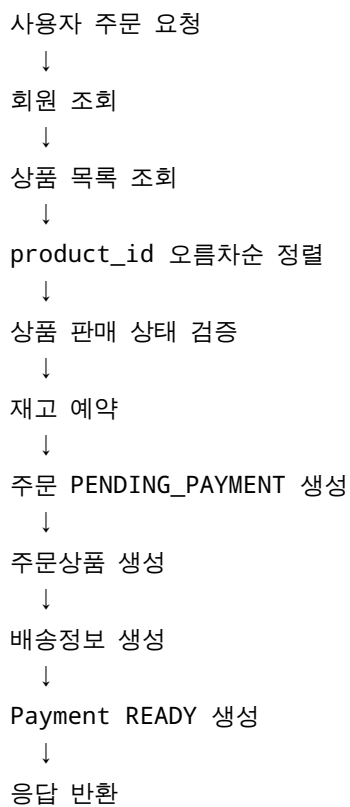
주문 생성에는 다음 작업이 포함된다.

1. 회원 조회
2. 상품 조회
3. 상품 판매 상태 검증
4. 재고 예약
5. 주문 생성
6. 주문상품 생성
7. 배송정보 생성
8. 결제 READY 생성
9. 트랜잭션 커밋

주문 생성은 하나의 트랜잭션으로 묶는다.

외부 PG 호출은 이 트랜잭션 안에서 수행하지 않는다.

8-2. 주문 생성 흐름



8-3. 주문 생성 Service 초안

```
@Transactional
public OrderCreateResponse createOrder(Long memberId, OrderCreateRequest request) {
    Member member = memberRepository.findById(memberId)
        .orElseThrow(() -> new IllegalArgumentException("회원이 존재하지 않습니다."));

    List<OrderCreateRequest.Item> sortedItems = request.getItems().stream()
        .sorted(Comparator.comparing(OrderCreateRequest.Item::getProductId))
        .toList();

    Order order = Order.createPending(member);

    for (OrderCreateRequest.Item item : sortedItems) {
        Product product = productRepository.findById(item.getProductId())
            .orElseThrow(() -> new IllegalArgumentException("상품이 존재하지 않습니다."));

        product.validateSelling();

        inventoryService.reserve(product.getId(), item.getQuantity());

        OrderItem orderItem = OrderItem.create(
            product.getId(),
            product.getName(),
            product.getPrice(),
            item.getQuantity()
        );

        order.addOrderItem(orderItem);
    }

    Delivery delivery = Delivery.create(request.getDeliveryAddress());
    order.setDelivery(delivery);

    Payment payment = Payment.ready(order, order.calculateFinalPaymentAmount());

    orderRepository.save(order);
    paymentRepository.save(payment);

    return OrderCreateResponse.from(order, payment);
}
```

9. 재고 처리 구현 방향

9-1. Java에서 재고를 단순 차감하지 않는다

다음 방식은 위험하다.

```
product.decreaseStock(quantity);
```

이 방식은 동시에 여러 사용자가 주문할 때 Race Condition이 발생할 수 있다.

재고 차감 또는 예약은 DB 조건부 UPDATE로 처리한다.

9-2. 재고 예약 SQL

```
UPDATE inventory
SET reserved_quantity = reserved_quantity + :quantity,
    available_quantity = available_quantity - :quantity
WHERE product_id = :productId
AND available_quantity >= :quantity;
```

변경된 row 수로 성공 여부를 판단한다.

```
updated row = 1 → 예약 성공
updated row = 0 → 재고 부족
```

9-3. InventoryService 초안

```
@Transactional
public void reserve(Long productId, int quantity) {
    int updated = inventoryRepository.reserve(productId, quantity);

    if (updated == 0) {
        throw new IllegalStateException("재고가 부족합니다.");
    }
}
```

10. 결제 구현 방향

1차 구현에서는 실제 PG사를 연동하지 않는다.
대신 Mock 결제 API를 만든다.

10-1. 결제 성공 흐름

```
결제 성공 요청
↓
payment READY 조회
↓
주문 PENDING_PAYMENT 조회
↓
결제 금액 검증
↓
payment APPROVED 변경
↓
order PAID 변경
↓
예약 재고 확정
```

10-2. 결제 실패 흐름

```
결제 실패 요청
↓
payment READY 조회
↓
payment FAILED 변경
↓
order PAYMENT_FAILED 변경
↓
예약 재고 해제
```

10-3. 결제 성공 Service 초안

```
@Transactional
public PaymentApproveResponse approve(PaymentApproveRequest request) {
    Payment payment = paymentRepository.findByPaymentKey(request.getPaymentKey())
        .orElseThrow(() -> new IllegalArgumentException("결제 정보가 없습니다."));

    Order order = orderRepository.findById(payment.getOrderId())
```

```

        .orElseThrow(() -> new IllegalArgumentException("주문 정보가 없습니다."));

    if (!order.isPendingPayment()) {
        return PaymentApproveResponse.alreadyProcessed(order, payment);
    }

    payment.approve(request.getAmount());
    order.pay();

    for (OrderItem item : order.getOrderItems()) {
        inventoryService.confirmReservedStock(item.getProductId(), item.getQuantity());
    }

    return PaymentApproveResponse.from(order, payment);
}

```

11. 주문 조회 구현 방향

주문 조회는 두 가지를 구분한다.

1. 주문 목록 조회
2. 주문 상세 조회

목록 조회에서는 모든 주문상품을 즉시 가져오지 않는다.
주문 목록에 필요한 최소 데이터만 조회한다.

상세 조회에서는 주문, 주문상품, 결제, 배송 정보를 함께 보여준다.

11-1. 주문 목록 조회 기준

```
GET /api/orders?cursor=...&size=20
```

조회 기준:

- member_id 기준
- ordered_at DESC
- order_id DESC
- cursor 기반 페이징

인덱스:

```
CREATE INDEX ix_orders_member_ordered
ON orders(member_id, ordered_at DESC, order_id DESC);
```

11-2. 주문 상세 조회 기준

주문 상세 조회에서는 다음 정보를 반환한다.

- 주문 기본 정보
- 주문상품 목록
- 결제 정보
- 배송 정보

주의할 점:

- 일반 사용자는 본인 주문만 조회 가능하다.
- Entity를 직접 반환하지 않는다.
- DTO로 변환해서 응답한다.

12. 리뷰 작성 구현 방향

리뷰는 실제 구매자만 작성할 수 있다.

리뷰 작성 조건은 다음과 같다.

1. 로그인한 회원이어야 한다.
2. order_item이 본인 주문에 속해야 한다.
3. 주문 상태가 DELIVERED 또는 CONFIRMED여야 한다.
4. 하나의 order_item에는 하나의 리뷰만 작성할 수 있다.

최종 중복 방지는 DB UNIQUE 제약조건으로 처리한다.

```
ALTER TABLE review
ADD CONSTRAINT uk_review_order_item UNIQUE(order_item_id);
```

13. 반드시 작성할 테스트

13-1. 도메인 테스트

```
Product.validateSelling()  
Order.pay()  
Order.cancel()  
Payment.approve()  
Payment.fail()  
Inventory.reserve()  
Inventory.release()
```

13-2. 통합 테스트

1. 재고가 충분하면 주문 생성 성공
2. 재고가 부족하면 주문 생성 실패
3. 주문 생성 성공 시 orders 저장
4. 주문 생성 성공 시 order_item 저장
5. 주문 생성 성공 시 delivery 저장
6. 주문 생성 성공 시 payment READY 저장
7. 결제 성공 시 order PAID 변경
8. 결제 성공 시 payment APPROVED 변경
9. 결제 성공 시 예약 재고 확정
10. 결제 실패 시 예약 재고 해제
11. 주문 취소 시 재고 복구
12. 구매하지 않은 상품 리뷰 작성 실패
13. 같은 order_item에 리뷰 중복 작성 실패

13-3. 동시성 테스트

동시성 테스트는 이 프로젝트의 핵심이다.

재고 1개 상품에 100명 동시 주문

기대 결과:

```
성공 주문 수 = 1  
실패 주문 수 = 99  
최종 available_quantity = 0  
재고 음수 발생 없음
```

같은 order_item에 리뷰 10개 동시 작성

기대 결과:

성공 리뷰 수 = 1
실패 리뷰 수 = 9
DB UNIQUE 제약조건으로 중복 방지

14. 성능 최적화 계획

14-1. 상품 목록 조회

상품 목록 조회는 다음 방식으로 최적화한다.

- 필요한 컬럼만 DTO로 조회
- category_id, status, created_at, product_id 복합 인덱스 사용
- Offset Pagination보다 Keyset Pagination 우선 고려

14-2. 주문 목록 조회

주문 목록 조회는 다음 방식으로 최적화한다.

- member_id 기준으로 조회
- ordered_at DESC, order_id DESC 정렬
- cursor 기반 페이징
- 목록 조회에서 order_item 전체를 즉시 로딩하지 않음

14-3. 리뷰 목록 조회

리뷰 목록 조회는 다음 방식으로 최적화한다.

- product_id 기준으로 조회
- deleted_at IS NULL 조건 적용
- created_at DESC, review_id DESC 기준 cursor pagination

15. README 작성 계획

GitHub README는 다음 구조로 작성한다.

```
# 운영 가능한 온라인 쇼핑몰 주문 시스템

## 1. 프로젝트 소개
## 2. 기술 스택
## 3. 핵심 목표
## 4. 시스템 아키텍처
## 5. ERD
## 6. API 명세
## 7. 주문 생성 흐름
## 8. 결제 성공/실패 흐름
## 9. 재고 동시성 제어
## 10. 주문 조회 성능 최적화
## 11. 테스트 전략
## 12. 동시성 테스트 결과
## 13. 트러블슈팅
## 14. 향후 개선 사항
```

README에서 가장 강조할 내용은 다음과 같다.

1. 단순 CRUD 쇼핑몰이 아니라 주문·결제·재고 정합성을 중심으로 설계했다.
2. 재고 차감은 DB 조건부 UPDATE로 처리했다.
3. 결제 성공/실패에 따라 주문 상태와 재고 상태를 일관되게 관리했다.
4. 동시성 테스트로 재고 음수 발생을 방지했다.
5. 조회 성능을 위해 인덱스와 cursor pagination을 적용했다.

16. 1차 구현 일정 초안

1주차

- 프로젝트 생성
- 패키지 구조 생성
- DB 연결
- Member, Product, Inventory Entity 작성
- 상품 등록/조회 API 구현

2주차

- Order, OrderItem, Delivery, Payment Entity 작성
- 주문 생성 Service 구현
- 주문 생성 API 구현
- 재고 예약 로직 구현

3주차

- Mock 결제 성공 API 구현
- Mock 결제 실패 API 구현
- 예약 재고 확정/해제 구현
- 주문 목록/상세 조회 구현

4주차

- 리뷰 작성 API 구현
- 리뷰 중복 방지 구현
- 주문/결제/재고 통합 테스트 작성
- 동시성 테스트 작성

5주차

- 상품 목록 조회 최적화
- 주문 목록 조회 최적화
- 리뷰 목록 조회 최적화
- 인덱스 적용 전/후 비교
- README 정리

17. 최종 구현 우선순위 요약

가장 먼저 할 일은 다음과 같다.

1. GitHub Repository 생성
2. Spring Boot 프로젝트 생성
3. member, product, inventory 테이블 작성
4. Member, Product, Inventory Entity 작성
5. 상품 등록/조회 API 구현
6. orders, order_item, delivery, payment 테이블 작성
7. Order, OrderItem, Delivery, Payment Entity 작성

- 8. 주문 생성 API 구현
- 9. 재고 예약 조건부 UPDATE 구현
- 10. Mock 결제 성공/실패 API 구현
- 11. 동시성 테스트 작성

18. 결론

지금 단계에서 중요한 것은 문서를 계속 확장하는 것이 아니다.
이미 운영 가능한 아키텍처 방향은 충분히 잡혔다.

이제 해야 할 일은 다음과 같다.

설계 문서
↓
MVP 범위 확정
↓
Entity 작성
↓
주문 생성 구현
↓
재고 동시성 테스트
↓
Mock 결제 흐름 구현
↓
조회 성능 최적화
↓
README와 포트폴리오 정리

이 프로젝트의 핵심 문장은 다음과 같다.

온라인 쇼핑몰의 핵심은 상품 CRUD가 아니라 주문·결제·재고의 정합성이다.

따라서 첫 번째 구현 목표는 상품 화면을 예쁘게 만드는 것이 아니라,
동시에 주문해도 재고가 깨지지 않는 주문 시스템을 만드는 것이다.